Test Plan: Digital Wallet Transfer Limits & Account Tiers

Document Version: 1.0
Author: QA Engineering - Solomon Eshiet
Date: 1 August 2026
Status: Draft — pending product team responses to open questions

---

1. Introduction

1.1 Feature Overview

This test plan covers **Digital Wallet Transfer Limits & Account Tiers**, a core payments capability within a mobile wallet application. The feature enables users to send money to other wallet users (wallet-to-wallet) and to external bank accounts (wallet-to-bank). Every account is assigned one of three tiers — Tier 1, Tier 2, or Tier 3 — determined by the level of identity verification (KYC) the user has completed.

Each tier imposes per-transaction and daily send limits. Tier 1 accounts may send up to 20,000 per transaction and 50,000 per day; Tier 2 accounts may send up to 200,000 per transaction and 500,000 per day; Tier 3 accounts have no per-transaction limit but are capped at 5,000,000 per day. The daily limit resets at midnight. Pending transfers count toward the daily limit, and amounts from failed or reversed transfers are returned to the user's available daily allowance. Transfers to a user's own linked bank accounts are exempt from all limits. Users may schedule transfers for a future date; scheduled transfers execute at 8:00 AM on the scheduled day. Tier upgrades take effect immediately upon completion of the next KYC level. Any transfer that would exceed the remaining daily limit is rejected with a clear error message showing the remaining limit.

1.2 Purpose of This Test Plan

The purpose of this document is to define a risk-based approach to testing the transfer limits and account tier logic before implementation test cases are executed. Because the feature specification contains ambiguities, contradictions, and unhandled edge cases typical of real-world product documents, this plan prioritises clarification of requirements, identification of high-risk failure modes, and a disciplined coverage strategy that avoids exhaustive but shallow testing.

This plan is intended to demonstrate engineering judgment: what to test first, what to automate, what to defer, and what must be resolved with the product team before reliable test oracles can be defined. It supports the companion Test Case Document (top 20 prioritised cases) and will be updated once open questions receive formal product responses.

1.3 Scope

In scope

- Per-transaction and daily send limit enforcement for Tier 1, Tier 2, and Tier 3 accounts
- Wallet-to-wallet and wallet-to-bank transfer types, including the own-linked-bank-account exemption
- Daily limit reset behaviour at midnight and its interaction with pending transfers
- Pending transfer counting, failure handling, and reversal credit back to available daily limit
- Scheduled transfer execution at 8:00 AM and limit validation at execution time
- Immediate tier upgrade upon KYC completion and its effect on in-flight transfers
- Rejection behaviour and error messaging when limits are exceeded, including remaining-limit display
- Concurrency scenarios where multiple transfers compete for the same remaining daily allowance
- Negative and boundary testing at tier-specific limit values

Out of scope

- End-to-end KYC vendor integration and document verification workflows (tier assignment is assumed via test fixtures unless otherwise noted)
- Inbound/receive-money flows, wallet funding sources, and general ledger reconciliation
- Multi-currency conversion and FX rate logic (currency is not defined in the specification; tests assume a single currency until clarified)
- Performance, load, and penetration testing of the payments infrastructure (covered separately if required)
- Regulatory reporting, AML monitoring rules, and audit trail completeness beyond limit-enforcement correctness
- UI visual design, accessibility certification, and localisation of all supported languages (error message clarity is in scope; full i18n matrix is not)
- Admin override, manual adjustment, and support tooling workflows unless product confirms they exist

---

2. Open Questions for Product Team

The following questions must be resolved before final test oracles can be signed off. Each item states the issue, why it matters for testing, and a specific question for the product team. Questions are grouped by category.

Category 1: Time, Timezone & Daily Reset

1.1 — Which timezone defines "midnight" for the daily limit reset?
Issue: "Resets at midnight" does not specify user local time, device timezone, account registration country, or a fixed UTC offset.
Why it matters: Two users (or one user travelling) could hit different reset boundaries for the same calendar day; scheduled and pending logic depends on when the counter zeroes.
Open question: Is the daily limit reset based on the user's profile timezone, device timezone, server UTC, or the country of the wallet account — and what happens when those disagree (e.g., user travels or changes device timezone)?

1.2 — Is the daily limit a calendar-day bucket or a rolling 24-hour window?
Issue: "Per day" and "midnight reset" imply calendar days, but many financial products use rolling windows.
Why it matters: Test data setup, boundary tests, and "remaining limit" calculations differ completely between models.
Open question: Is "50,000 per day" a fixed calendar day (reset at midnight) or a rolling 24-hour sum of completed and pending transfers?

1.3 — What timezone applies to the 8:00 AM scheduled-transfer execution time?
Issue: Rule 5 says scheduled transfers run at 8:00 AM on the scheduled day, but not whose 8:00 AM.
Why it matters: A transfer "scheduled for Monday" may execute at different absolute times; interacts with midnight reset and daily exhaustion.
Open question: Does "8:00 AM on the scheduled day" use the same timezone as the daily reset, and is that timezone locked at booking time or evaluated at execution time?

1.4 — How is "the scheduled day" defined across timezone boundaries?
Issue: User may schedule while in one timezone and execute after travel or DST change.
Why it matters: Off-by-one-day execution and wrong daily bucket assignment.
Open question: If a user schedules a transfer while in UTC+1 but their account timezone is UTC+0, which date is "the scheduled day" for execution and limit counting?

1.5 — How do Daylight Saving Time transitions affect midnight reset and 8:00 AM execution?
Issue: On DST spring-forward, "midnight" and "8:00 AM" may not exist or may occur twice.
Why it matters: Missing or duplicate executions; ambiguous limit reset moments.
Open question: What is the defined behaviour on DST transition days for daily reset and scheduled execution (e.g., skip, shift, or use UTC internally)?

Category 2: Pending Transfers & Midnight Straddling

2.1 — When a pending transfer straddles midnight, which day's limit does it consume?
Issue: Rule 3 says pending transfers count toward the daily limit, but not whether counting is by initiation day, execution day, or split across days.
Why it matters: Core boundary test: user at limit near midnight with a long-pending transfer.
Open question: If a transfer is initiated at 11:50 PM and still pending at 12:01 AM, does it count against the old day, the new day, both, or neither until it completes?

2.2 — At midnight reset, are in-flight pending amounts released from the old day and re-applied to the new day?
Issue: Pending amounts may still be reserved when the counter resets.
Why it matters: User could appear to have full daily limit while large pending transfers still exist, or double-count if not handled atomically.
Open question: At reset, is the pending portion of the daily tally cleared, carried forward, or re-evaluated against the new day's limit?

2.3 — What statuses count as "pending" for limit purposes?
Issue: Spec does not define pending vs processing vs authorised vs scheduled vs queued.
Why it matters: Different statuses may need different limit treatment; tests need exact state machine.
Open question: Which transfer statuses count toward the daily limit (e.g., scheduled-not-yet-run, authorised-awaiting-settlement, processing, on hold for fraud review)?

2.4 — If a pending transfer completes after midnight, which day's limit governed its approval?
Issue: Approval may have happened pre-reset but settlement post-reset (or vice versa).
Why it matters: Determines whether a transfer should have been rejected and whether limits were enforced correctly.
Open question: Is limit enforcement evaluated at initiation, at execution/settlement, or re-checked at both — and which timestamp wins if the day boundary falls between them?

Category 3: Concurrency & Race Conditions

3.1 — How are two simultaneous transfers handled when their combined amount exceeds remaining daily limit?
Issue: No locking, ordering, or "first wins" rule is specified.
Why it matters: Classic race: two requests each see enough remaining limit; both succeed and exceed cap.
Open question: When two transfers are submitted concurrently against the same remaining daily limit, is enforcement optimistic, pessimistic (serialized with one rejection), or based on arrival order — and is that guarantee documented for API consumers?

3.2 — Is daily limit reservation atomic at request time?
Issue: Unclear whether limit is decremented on submit, on auth, or on settlement.
Why it matters: Defines when races occur and what "remaining limit" in the error message reflects.

Open question: At what exact moment is the daily limit decremented/reserved, and is that reservation held until completion, failure, or timeout?

3.3 — What does the "remaining limit" in the rejection error reflect under concurrent load?
Issue: Rule 7 requires showing remaining limit, but remaining may change between read and reject.
Why it matters: Error message accuracy and user trust; flaky tests if value is non-deterministic under load.
Open question: Is the remaining limit in the error message a point-in-time snapshot at rejection, and could it differ from what another concurrent request observed milliseconds earlier?

3.4 — Are retries/idempotent replays treated as new limit consumption or deduplicated?
Issue: Network retries could double-reserve or double-count the same transfer intent.
Why it matters: Duplicate charges against limit without duplicate successful transfers.
Open question: If a client retries the same transfer request (same idempotency key or duplicate tap), does it consume the daily limit once or multiple times until deduplication resolves?

Category 4: Failures, Reversals & Post-Reset Recovery

4.1 — When exactly is failed/reversed amount returned to available daily limit?
Issue: Rule 3 says amount is "returned" but not synchronously, after settlement, or after a delay.
Why it matters: User may retry immediately after failure and still see exhausted limit.
Open question: Is the daily limit credit applied immediately on failure/reversal confirmation, and is that visible in real time in the UI/API?

4.2 — If a reversal arrives after the daily reset, which day's limit is credited?
Issue: Original transfer may have consumed yesterday's bucket; reversal may land today.
Why it matters: User could gain extra send capacity or lose expected credit.
Open question: When a transfer initiated on Day N is reversed on Day N+1, is the credited amount added to Day N+1's available limit, Day N's (if still open), or handled as a separate adjustment outside daily limits?

4.3 — Are partial reversals supported, and how do they affect daily limit?
Issue: Full reversal vs partial chargeback/refund not specified.
Why it matters: Remaining limit math and reconciliation tests.
Open question: If only part of a transfer is reversed, is the daily limit restored by the reversed portion only, and can the unreversed portion still count against the original day?

4.4 — What happens if a transfer fails after partially consuming limit (e.g., timeout unknown state)?
Issue: Ambiguous final state may leave limit reserved indefinitely.
Why it matters: "Stuck" limit reduces available balance until manual fix.
Open question: What is the timeout/reconciliation behaviour for transfers in unknown state, and when is reserved limit automatically released?

4.5 — Do chargebacks/disputes on completed transfers follow the same "return to daily limit" rule as internal failures?
Issue: Rule 3 mentions "reversed" but not external bank chargebacks days later.
Why it matters: Long-tail limit accounting and regulatory expectations.
Open question: Are bank-initiated chargebacks treated like reversals for daily limit restoration, and if so, on which day's limit?

Category 5: Scheduled Transfers & Limit Exhaustion

5.1 — What happens when a scheduled transfer executes on a day where the daily limit is already exhausted?
Issue: No behaviour defined for execution-time rejection vs queue vs partial execution.
Why it matters: High-risk scenario: money movement when it should not happen, or failed expected payments.
Open question: If a scheduled transfer fires at 8:00 AM but the user has already used their full daily limit, is it rejected, retried later, rescheduled, or executed anyway because it was booked earlier?

5.2 — Is limit checked at schedule time, execution time, or both?
Issue: User could schedule many future transfers that collectively exceed any future day's limit.
Why it matters: Determines whether scheduling itself can fail under Rule 7.
Open question: When a user schedules a transfer, do we validate against today's limit, the scheduled day's projected limit, or skip validation until execution?

5.3 — Do scheduled transfers reserve daily limit before execution day?
Issue: If not reserved, multiple schedules could all pass validation; if reserved early, they reduce today's limit.
Why it matters: Fundamentally different test scenarios for multi-day scheduling.
Open question: From the moment a transfer is scheduled, does it count toward the daily limit of the booking day, the execution day, or only when it runs at 8:00 AM?

5.4 — What does "per transaction" mean for a scheduled transfer booked in advance?
Issue: Tier per-transaction caps may apply at booking or at execution when limits/tier may differ.
Why it matters: User schedules 300,000 as Tier 2 (max 200,000 per transaction) — valid at booking?
Open question: For scheduled transfers, are per-transaction and daily limits evaluated using tier and limits at schedule time, at execution time, or whichever is stricter?

5.5 — Can users schedule multiple transfers on the same day that together exceed the daily limit?
Issue: No aggregate validation across scheduled items.
Why it matters: All may fail at 8:00 AM or race at execution.
Open question: Is there a cap on total scheduled outflow per execution day, and how are conflicts between multiple 8:00 AM jobs on the same day resolved?

5.6 — Can scheduled transfers be edited or cancelled after booking, and how does that affect limit accounting?
Issue: Changes to amount/date/tier context not covered.
Why it matters: Limit reservations may become stale.
Open question: If a user increases a scheduled amount or changes the date, is limit re-validated and are prior reservations released?

Category 6: Tier Upgrades, Downgrades & In-Flight Transfers

6.1 — What happens if a tier upgrade occurs while a transfer is in-flight?
Issue: Rule 6 says upgrade is immediate, but not whether in-flight transfers re-evaluate against new tier limits.
Why it matters: Transfer initiated under Tier 1 caps may complete after upgrade to Tier 3, or be rejected mid-flight.
Open question: For a transfer already pending/processing when KYC completes, are limits re-checked against the new tier, and can a previously rejected transfer become valid after upgrade?

6.2 — Can tier downgrade occur (KYC expiry, fraud, admin action), and what is the effective timing?
Issue: Spec only covers upgrade "immediately"; downgrade not mentioned.
Why it matters: User at Tier 2 with 400,000 sent today could drop to Tier 1 mid-day.
Open question: If a user's KYC level is revoked or expires, does tier downgrade take effect immediately, and how are in-flight transfers and remaining daily usage handled?

6.3 — If downgrade makes today's usage exceed the new tier's daily limit, is sending blocked retroactively?
Issue: Already-completed transfers may exceed new caps.
Why it matters: Grandfathering vs hard enforcement affects regression and edge tests.
Open question: After downgrade, do we block new transfers only, or also flag/reverse transfers that exceeded the new tier limits while they were still valid under the old tier?

6.4 — Does Tier 3 "no per-transaction limit" still imply a practical maximum (balance, fraud cap, API max)?
Issue: Unlimited per transaction is rarely truly unlimited in systems.
Why it matters: Boundary tests for Tier 3 need an upper bound or explicit "no max."
Open question: For Tier 3, is there any system-level maximum per transfer (e.g., available balance, fraud threshold), or is it literally unbounded aside from the 5,000,000 daily cap?

Category 7: Transfer Types, Exemptions & "Own Linked Bank Account"

7.1 — Are wallet-to-wallet and wallet-to-bank transfers subject to the same tier and daily limits?
Issue: Limits are stated globally; exemption in Rule 4 applies only to own linked bank accounts.
Why it matters: Matrix coverage: transfer type × tier × limit state.

Open question: Do transfers to other users' wallets count toward per-transaction and daily limits the same way as transfers to third-party bank accounts?

7.2 — How is "user's own linked bank account" verified, and can it be spoofed or confused with similar accounts?
Issue: Exemption from all limits is a high-abuse surface; verification method unspecified.
Why it matters: Security and limit-bypass testing; false positive/negative linking.
Open question: What criteria mark a bank account as "own linked" (name match, verified micro-deposit, account token from bank OAuth), and what prevents a user from linking a third-party account that appears to be theirs?

7.3 — Does the exemption apply only to outbound transfers to linked accounts, or also inbound pulls, wallet top-ups, or internal "move money" between wallet and linked bank?
Issue: "Transfers to" is direction-specific; other flows may circumvent limits.
Why it matters: Users might route large amounts via exempt paths.
Open question: Are inbound transfers from a linked bank account, wallet-to-wallet via linked account routing, or "transfer to self" between two linked accounts exempt from limits?

7.4 — What happens if a linked bank account is unlinked or re-linked while transfers are pending or scheduled?
Issue: Exemption depends on link state at unknown point in lifecycle.
Why it matters: Transfer may flip from exempt to limited mid-flight.
Open question: At which moment is linked-account status evaluated for exemption — schedule time, execution time, or initiation time — and what happens to pending exempt transfers if the link is removed?

7.5 — Are transfers to the user's own wallet (same user, different wallet ID) or between a user's multiple profiles treated as exempt or limited?
Issue: "Own linked bank account" does not mention self-wallet transfers.
Why it matters: Potential limit bypass or unexpected enforcement.
Open question: Is sending money to another wallet owned by the same KYC identity exempt from limits, or treated like any wallet-to-wallet transfer?

Category 8: Currency, Amounts & Partial Transfers

8.1 — What currency (or currencies) do the numeric limits apply to?
Issue: Spec uses bare numbers (20,000; 50,000) with no ISO currency code or multi-currency rules.
Why it matters: FX conversion, rounding, and limit comparison all depend on currency model.
Open question: Is this a single-currency product only, and if multi-currency is supported, are limits enforced in wallet currency, sender currency, or a base currency at spot rate?

8.2 — Are fractional amounts allowed, and what rounding rules apply near limits?
Issue: No precision (e.g., cents/kobo) or rounding direction specified.
Why it matters: Off-by-one-cent acceptance/rejection at boundaries.

Open question: What is the minimum transfer increment, and when comparing against limits, is amount rounded up, down, or truncated?

8.3 — Are partial transfers allowed when the requested amount exceeds remaining daily limit but is within per-transaction limit?
Issue: Rule 7 says reject if transfer would exceed remaining limit; does not mention partial execution.
Why it matters: UX and API contract: reject all vs send max available.
Open question: If a user requests 10,000 but only 7,000 daily limit remains, is the transfer fully rejected, or can the user/system execute a partial 7,000 transfer?

8.4 — What is the minimum and maximum transfer amount (if any) independent of tier?
Issue: Zero, negative, and micro-transfers not addressed.
Why it matters: Validation and abuse tests.
Open question: Are zero-amount or negative-amount transfers rejected, and is there a global minimum transfer amount below tier limits?

8.5 — For Tier 3 with no per-transaction limit, is a single transfer of 5,000,000 allowed if it equals the full daily cap?
Issue: Interaction between "no per-transaction limit" and daily cap unclear for exact-boundary cases.
Why it matters: Boundary test at daily max in one transaction.
Open question: Can Tier 3 users send their entire 5,000,000 daily allowance in one transaction, and does that exhaust the day completely?

Category 9: Error Handling, UX & Rule Consistency

9.1 — What is the exact format and precision of the "remaining limit" shown in rejection errors?
Issue: Rule 7 requires a "clear" message with remaining limit but no format, locale, or currency display rules.
Why it matters: Localization, accessibility, and automated assertion of error payloads.
Open question: Should the error include remaining daily limit only, or also remaining per-transaction headroom, tier name, and reset time — and in what numeric/currency format?

9.2 — Is there a contradiction between Rule 4 (exempt transfers) and Rule 7 (reject when exceeding limit)?
Issue: Rule 7 appears global; Rule 4 exempts a class of transfers — precedence not stated.
Why it matters: Tests must know which rule wins for exempt paths near limit.
Open question: For transfers to own linked bank accounts, is Rule 7 skipped entirely, or is remaining limit still calculated but not enforced?

9.3 — Does "rejected" mean hard failure with no retry, or soft failure with option to reduce amount?
Issue: Rejection behaviour beyond message not defined.

Why it matters: Client flows and idempotent retry semantics.
Open question: When a transfer is rejected for limit exceeded, can the client immediately retry with a lower amount using the same session, or is cooldown/duplicate detection applied?

9.4 — Are incoming transfers (money received) subject to any limits or affect outbound remaining limit?
Issue: Spec focuses on "send" limits; receiving side omitted.
Why it matters: Balance vs limit confusion; inbound may fund outbound same day.
Open question: Do received transfers increase available balance only, or also affect daily send limit calculations or tier usage reporting?

Category 10: Scope Gaps & Operational Edge Cases

10.1 — What defines "identity verification (KYC) completed" for each tier transition?
Issue: Tier assignment depends on KYC level but levels and criteria are undefined.
Why it matters: Cannot test tier boundaries without KYC state fixtures.
Open question: What are the exact KYC requirements for Tier 1 vs 2 vs 3, and can a user be Tier 2 for limits but pending review for a higher tier?

10.2 — Are limits enforced per wallet account, per user identity, or per device?
Issue: One user might hold multiple wallets or joint accounts.
Why it matters: Limit aggregation and test user setup.
Open question: If one person has multiple wallet accounts, are daily limits shared at the identity level or independent per account?

10.3 — What happens when the account is suspended, under fraud review, or has insufficient balance but within limit?
Issue: Limit is only one rejection reason; precedence with other blocks unclear.
Why it matters: Error message priority and test oracles.
Open question: When multiple failure reasons apply (insufficient balance, limit exceeded, account frozen), which error is returned and is limit still shown?

10.4 — Is there an admin/adjustment path that overrides limits, and is that auditable?
Issue: Support overrides common in wallet products but not in spec.
Why it matters: Tests may need to exclude admin actions or simulate them.
Open question: Can support or admin users execute transfers above tier/daily limits, and if so, are those transfers excluded from user-facing limit calculations?

10.5 — How are limits communicated proactively before the user submits a transfer?
Issue: Rule 7 is reactive (on rejection); no requirement for upfront display.
Why it matters: UI/API tests for remaining limit display vs enforcement consistency.
Open question: Must the app show remaining daily and per-transaction limits before submission, and is that value guaranteed to match enforcement logic at submit time?

---

3. Risk-Based Test Prioritisation

Testing for this feature is prioritised by financial and regulatory impact, likelihood of implementation defect, and difficulty of detection in production. The specification defines limit rules that appear straightforward in isolation but interact across time boundaries, asynchronous transfer states, tier changes, and exemption paths. A defect in any of these interaction points can allow users to send money above their permitted tier cap — a failure mode with direct compliance exposure — or block legitimate transfers, causing customer harm and operational cost.

The highest-risk scenarios are not the static boundary values (for example, rejecting 20,001 on a Tier 1 account), which are relatively easy to implement and detect. The critical risk concentrates where multiple rules apply simultaneously: a pending transfer approaching midnight, a scheduled job firing into an exhausted daily bucket, two concurrent requests reading the same remaining limit before either writes, or an exemption evaluated at the wrong point in the transfer lifecycle. These defects are often intermittent, environment-dependent, and invisible to shallow happy-path regression unless tests are designed explicitly to expose them.

The following five scenarios represent the top of the risk register and will receive earliest test design and execution effort once environment prerequisites are met.

Rank	Scenario	Why it is high risk
1	Two simultaneous transfers racing the same remaining daily limit	**Financial & regulatory:** If both requests succeed, the user exceeds their tier cap — a direct limits-control failure. **Likelihood:** Concurrency bugs are common when limit checks are read-then-write rather than atomic. **Detection:** Often invisible in manual testing; may only surface under production load or duplicate-tap behaviour.
2	Pending transfer straddling the midnight daily reset	**Financial:** Incorrect counting can let users send above the daily cap or block legitimate transfers. **Likelihood:** Time-boundary logic is a frequent defect class, especially with timezones and DST. **Detection:** Intermittent; fails only in a narrow time window unless time is mocked.
3	Scheduled transfer executing at 8:00 AM when the day's limit is already exhausted	**Financial & trust:** Silent failure of expected payments damages user trust; unintended execution over the limit is a compliance failure. **Likelihood:** Batch/scheduler logic often diverges from real-time limit enforcement. **Detection:** Requires orchestrating state across days and jobs.

4	Limit bypass via "own linked bank account" exemption (false linkage or wrong evaluation timing)	**Financial & fraud:** Rule 4 exempts all limits — a single verification flaw allows unlimited outbound flow. **Regulatory:** Highest-severity abuse vector in this spec. **Detection:** Requires negative security testing, not just functional happy paths.
5	Tier change (upgrade or downgrade) while a transfer is in-flight	**Financial:** A transfer approved under Tier 1 caps may complete after upgrade to Tier 3 (under-enforcement), or a valid Tier 2 transfer may fail after downgrade. **Likelihood:** KYC completion is async to payment rails. **Detection:** Needs multi-system coordination and timing control.

Execution order follows this risk ranking. P0 test cases (documented separately) cover core limit enforcement for all three tiers, midnight reset behaviour, and pending transfer accounting including failure and reversal. P1 cases address the exemption path, tier upgrade during in-flight transfer, and scheduled transfer limit checks at execution. P2 cases cover concurrent race conditions and error message accuracy, which are important but depend on stable core enforcement before they can be interpreted correctly.

Where the specification is ambiguous (see Section 2), tests are written with the best available oracle and flagged as blocked or conditional until product confirmation. This is intentional: executing tests against guessed behaviour creates false confidence; documenting the dependency demonstrates appropriate QA gatekeeping.

---

4. Coverage Strategy

4.1 Combination space

The feature varies across three primary dimensions: account tier (Tier 1, Tier 2, Tier 3), transfer type (wallet-to-wallet, wallet-to-bank), and limit state (under limit, at boundary, over limit / rejected, and exempt for own linked bank accounts). A naive full cross of these dimensions yields 3 × 2 × 4 = **24 combinations** before any edge-case dimension (pending, scheduled, midnight, concurrency, tier-in-flight) is introduced. Crossing every edge case against every combination would produce several hundred test cases with diminishing returns.

4.2 Pairwise reduction rationale

The coverage strategy applies four reduction rules to reach approximately **15 core matrix cases plus 12 targeted edge-case scenarios (~27 total)** without sacrificing meaningful risk coverage.

Equivalence partitioning. "Under limit" is treated as a single equivalence class per tier — any amount comfortably below the cap exercises the same code path. Only **at-boundary** and **over-limit** values require tier-specific amounts (e.g., 20,000 / 20,001 for Tier 1; 200,000 / 200,001 for Tier 2; 5,000,000 daily cap for Tier 3).

Risk-weighted tier sampling. Tier 1 receives full depth because its tight caps (20,000 / 50,000) create the highest boundary precision risk. Tier 2 receives spot-checks to prove limit scaling without repeating every permutation. Tier 3 focuses on the unique absence of a per-transaction limit and enforcement of the 5,000,000 daily ceiling only.

Constraint-driven omission. The exempt limit state applies **only** to wallet-to-bank transfers to the user's own linked bank account. Exempt is not crossed with wallet-to-wallet transfers because Rule 4 does not apply to that path.

Pairwise (all-pairs) coverage. Selected cases ensure every pair of (Tier, Transfer type), (Tier, Limit state), and (Transfer type, Limit state) appears at least once, with priority given to boundary, over-limit, and exempt cells.

4.3 Coverage matrix (selected cases)

ID	Tier	Transfer Type	Limit State	Test Intent	Rationale
M-01	T1	Wallet	Under	Happy path within caps	Baseline for smallest tier; regression anchor
M-02	T1	Wallet	At boundary	Tx = 20,000; daily = 50,000	Highest precision risk at lowest caps
M-03	T1	Wallet	Over	Tx = 20,001 → reject + remaining limit	Core Rule 7 enforcement
M-04	T1	Bank (third party)	At boundary	Daily boundary on bank rail	Pairwise: bank × T1 × boundary
M-05	T1	Bank (own linked)	Exempt	Amount > T1 daily cap → succeeds	Rule 4 — only applicable exempt cell
M-06	T2	Wallet	At boundary	Tx = 200,000; daily = 500,000	Spot-check tier scaling
M-07	T2	Wallet	Over	Tx = 200,001 → reject	Pairwise: T2 × over
M-08	T2	Bank (third party)	Under	Happy path	Pairwise: T2 × bank × under

M-09	T3	Wallet	Under	Large tx > T2 per-tx cap (e.g., 300,000)	Unique T3: no per-transaction limit
M-10	T3	Wallet	At boundary	Single tx = 5,000,000 (daily max)	Daily cap is only guardrail
M-11	T3	Wallet	Over	Daily total > 5,000,000 → reject	T3 over-limit is daily-only
M-12	T3	Bank (third party)	Over	Same as M-11 on bank rail	Pairwise: T3 × bank × over
M-13	T2	Bank (own linked)	Exempt	Exempt on mid-tier	Exemption independent of tier
M-14	T1	Bank (third party)	Over	Daily exhausted, tx within per-tx cap	Distinguishes per-tx vs daily rejection
M-15	T3	Bank (own linked)	Exempt	High-value exempt transfer	Pairwise: T3 × exempt

Documented omissions (not forgotten): Tier 2/Tier 3 "under limit" wallet paths beyond M-08/M-09 are covered by equivalence class sampling. Exempt × wallet-to-wallet is N/A per Rule 4.

4.4 Edge-case suites (outside the core matrix)

Edge scenarios are not fully crossed against the matrix. Each suite receives one to three targeted cases:

Suite	Focus	Approx. Cases
E1 — Time	Midnight reset; pending straddles reset; 8 AM scheduled execution timezone	3–4
E2 — Concurrency	Parallel transfers vs remaining limit; idempotent retry	2
E3 — Lifecycle	Pending counts toward limit; fail/reversal restores limit; reversal after reset	3
E4 — Tier transition	Upgrade mid-flight; downgrade with usage above new cap	2

E5 — Scheduling	Schedule when under limit, execute when exhausted; limit check at book vs run	2

---

5. Automation vs Manual Testing Strategy

The split between automated and manual testing for this feature is driven by determinism, regression value, and the need for human judgment. Limit enforcement logic — per-transaction caps, daily tallies, pending reservation, failure/reversal credit, and rejection payloads — is inherently rule-based and stable once requirements are confirmed. These behaviours are exposed at the API layer, produce binary pass/fail outcomes, and are exactly the class of checks that should run on every build to prevent financial-control regressions. For this reason, approximately **70 percent of executed verification** is allocated to automated API regression, with a further **10 percent** for targeted UI smoke tests that confirm limit errors surface correctly to the user.

Automated coverage prioritises, in order: limit enforcement and error contract validation (reject when over per-transaction or daily cap; assert remaining limit in response); tier boundary values via parameterised data-driven tests; daily tally arithmetic including pending, failure, and reversal; exempt-path positive and negative cases (own linked account bypasses limits; third-party bank does not); concurrent limit races using controlled parallel requests; and time-boundary behaviour using clock injection for midnight reset and 8:00 AM scheduled execution. Tests will create and clean up their own transfer data to remain deterministic and repeatable. Any test whose oracle depends on an unresolved open question (Section 2) will be marked conditional and skipped in CI until product confirmation is received.

Manual and exploratory testing retains approximately **20 percent of effort** for scenarios where human judgment, adversarial thinking, or environmental fidelity cannot be cost-effectively automated within assessment scope. This includes timezone and DST behaviour across user profile, device, and server; qualitative assessment of error message clarity and locale formatting; end-to-end KYC tier upgrade journeys involving third-party verification UI; linked-account verification and spoofing attempts; scheduled transfer failure UX (notifications, retry, reschedule prompts); and exploratory probing of spec gaps such as partial transfer offers, multi-schedule same-day conflicts, and error precedence when insufficient balance and limit exceeded co-occur.

The handoff between manual and automated testing is explicit: any defect or ambiguity discovered during exploratory sessions becomes a candidate for automation once the product team resolves the underlying open question. Manual testing finds; automation prevents regression. UI automation is deliberately scoped to smoke depth — login, submit transfer, verify

limit error visible — because the financial correctness oracle lives at the API layer and flaking UI tests on a shared demo environment would add noise without improving limit-logic confidence.

---

6. Test Environment Assumptions

Effective verification of transfer limits and tier logic cannot be performed against production or an undocumented shared environment. The following assumptions define the minimum test environment required to execute this plan with confidence.

API access and test accounts. The test environment must expose stable API endpoints for transfer initiation, transfer status query, daily limit/usage query, account tier lookup, and linked-bank-account management. Separate test accounts (or provisionable fixtures) are required for Tier 1, Tier 2, and Tier 3, each pre-funded with sufficient balance to reach tier daily caps (minimum 5,000,000 available for Tier 3 scenarios). Accounts must support configurable linked bank accounts, including at least one verified "own linked" account and one third-party bank recipient for exemption contrast testing.

Time manipulation. Tests for midnight reset (TC-008), pending straddling midnight (TC-009), and scheduled 8:00 AM execution (TC-016, TC-017) require either a test environment with injectable/mockable system time or a dedicated scheduler test hook that allows triggering scheduled jobs without waiting wall-clock time. Without time control, P0 time-boundary cases cannot be executed deterministically and will remain manual or blocked.

Transfer state control. The environment must support inducing transfer states — Pending, Completed, Failed, Reversed — without manual database edits where possible. This includes a simulated failure endpoint or invalid recipient for failure-path tests (TC-011) and a reversal/refund mechanism for TC-012. If the payments sandbox cannot fail on demand, a test-only admin API or fixture reset between runs is required.

Concurrency testing infrastructure. Race-condition tests (TC-018) require the ability to submit at least two transfer requests in parallel against the same account from independent sessions, with millisecond-level timing control. A single-threaded UI cannot reliably reproduce this; API-level parallel execution (e.g., async HTTP clients or load tool with two virtual users) is assumed.

KYC/tier transition simulation. Tier upgrade mid-flight tests (TC-015) require a test hook to complete KYC and promote tier without manual back-office intervention, or a stubbed identity service that responds on demand. Downgrade scenarios (E4) depend on product confirmation that downgrade exists.

Data isolation and cleanup. Each automated test run must start from a known daily spend state (zero or explicitly seeded). Tests create their own transfer data and clean up afterward so

repeated runs do not accumulate spend against daily limits. A daily limit reset API or account recycle mechanism is assumed for CI pipelines.

Single currency. Until open question 8.1 is resolved, the environment is assumed to operate in a single currency with whole-unit amounts (no fractional kobo/cent boundary ambiguity).

Observability. Logs or audit trails must be available to diagnose limit reservation timing, scheduler execution, and concurrent request ordering when tests fail. This is essential for triaging race and midnight-boundary defects that are not visible from the user-facing response alone.

If any of the above assumptions cannot be met, affected test cases will be documented as blocked in the Test Case Document with the specific environment gap noted. Priority will shift to API-level cases that can run with available tooling while open questions and environment dependencies are resolved.
